

EDTECH API DOCUMENTATION

Base URL: <http://localhost:5001>

Authentication: JWT Bearer Token (for protected routes)

TABLE OF CONTENTS

1. 1. Authentication Routes
2. 2. Child Profile Routes
3. 3. Subject Routes
4. 4. Chapter Routes
5. 5. Payment Routes
6. 6. Quiz Question Routes
7. 7. Quiz Score Routes
8. 8. Adaptive Quiz Routes
9. 9. Leaderboard Routes
10. 10. Health Check

1. AUTHENTICATION ROUTES

1.1 Parent Login - Send OTP

POST /api/v1/parent/login

Authentication: None

Description: Sends OTP to parent's email for login/registration

Request Body (JSON):

- ```
{
 "email": "parent@example.com"
}
```

#### Success Response:

- ```
Success Response (200):
{
  "message": "OTP Sent Successfully",
  "isNewUser": false
}
```

For New User:

```
{
  "message": "Account created! OTP Sent Successfully",
  "isNewUser": true
}
```

Error Responses:

- ```
Error (400):
{
 "message": "Email is required"
}
```

## 1.2 Verify Parent OTP

**POST** /api/v1/verify/parent

**Authentication:** None

**Description:** Verifies OTP and returns JWT token

### Request Body (JSON):

- ```
{
  "email": "parent@example.com",
  "otp": "123456"
}
```

Success Response:

- Success Response (200):

```
{
  "message": "Parent Login Success",
  "token": "eyJhbGc...JWT_TOKEN",
  "user": {
    "id": "67890abcdef123456",
    "email": "parent@example.com",
    "role": "parent",
    "isParent": true
  }
}
```

Error Responses:

- Error (400):

```
{
  "message": "Invalid OTP"
}
```
- Error (400):

```
{
  "message": "User not found"
}
```

1.3 Check User Exists

POST /api/v1/user/check

Authentication: None

Description: Checks if user exists in database

Request Body (JSON):

- ```
{
 "email": "parent@example.com"
}
```

#### Success Response:

- User Exists (200):

```
{
 "message": "User exists",
 "exists": true
}
```
- User Not Found (404):

```
{
 "message": "User not found",
 "exists": false
}
```

### 1.4 Get Current User Info

#### GET /api/v1/me

**Authentication:** Required (Bearer Token)

**Description:** Returns authenticated user's information

#### Headers:

- Authorization: Bearer <JWT\_TOKEN>

#### Success Response:

- Success Response (200):

```
{
 "id": "67890abcdef123456",
 "email": "parent@example.com",
 "name": "John Parent",
 "role": "parent",
 "isParent": true
}
```

#### Error Responses:

- Error (401):

```
{
 "message": "Unauthorized"
}
```

### 1.5 Check Parent Session

#### GET /api/v1/parent/check-session

**Authentication:** Required (Bearer Token - Parent only)

**Description:** Checks if parent session requires OTP verification

### Headers:

- Authorization: Bearer <JWT\_TOKEN>

### Success Response:

- Active Session (200):

```
{
 "requiresOtp": false,
 "lastActivity": "2026-01-10T10:30:00.000Z",
 "timeDiffDays": 3.5
}
```
- Expired Session (200):

```
{
 "requiresOtp": true,
 "lastActivity": "2025-12-01T10:30:00.000Z",
 "timeDiffDays": 15
}
```

## 2. CHILD PROFILE ROUTES

### 2.1 Get Parent's Children

**GET** /api/v1/parent/children

**Authentication:** Required (Bearer Token - Parent only)

**Description:** Returns all children profiles for authenticated parent

### Headers:

- Authorization: Bearer <JWT\_TOKEN>

### Success Response:

- Success Response (200):

```
{
 "children": [
 {
 "id": "67890child001",
 "name": "Emma",
 "email": null,
 "classno": 1,
 "emoji": "🐱"
 },
 {
 "id": "67890child002",
 "name": "Noah",
 "classno": 2,
 "emoji": "🐶"
 }
]
}
```

## 2.2 Create Child Profile

**POST** /api/v1/parent/child

**Authentication:** Required (Bearer Token - Parent only)

**Description:** Creates a new child profile

### Headers:

- Authorization: Bearer <JWT\_TOKEN>

### Request Body (JSON):

- ```
{
  "name": "Emma",
  "classno": 1,
  "emoji": "🐱"
}
```

Success Response:

- Success Response (201):

```
{
  "message": "Child profile created successfully",
  "child": {
    "id": "67890child001",
    "name": "Emma",
    "classno": 1,
    "emoji": "🐱"
  }
}
```

Error Responses:

- Error (400):

```
{
  "message": "Name and class are required"
}
```

2.3 Update Child Profile

PUT /api/v1/parent/child/:childId

Authentication: Required (Bearer Token - Parent only)

Description: Updates existing child profile

Headers:

- Authorization: Bearer <JWT_TOKEN>

URL Parameters:

- childId - MongoDB ObjectId of the child

Request Body (JSON):

- ```
{
 "name": "Emma Updated",
 "classno": 2,
 "emoji": "👩"
}
```

#### Success Response:

- Success Response (200):

```
{
 "message": "Child profile updated successfully",
 "child": {
 "id": "67890child001",
 "name": "Emma Updated",
 "classno": 2,
 "emoji": "👩"
 }
}
```

#### Error Responses:

- Error (404):

```
{
 "message": "Child profile not found"
}
```

## 2.4 Delete Child - Send OTP

### POST /api/v1/parent/child/delete-otp

**Authentication:** Required (Bearer Token - Parent only)

**Description:** Sends OTP to parent's email for child deletion confirmation

#### Headers:

- Authorization: Bearer <JWT\_TOKEN>

#### Request Body (JSON):

- ```
{
  "childId": "67890child001"
}
```

Success Response:

- Success Response (200):

```
{
  "message": "OTP sent successfully"
}
```

Error Responses:

- Error (404):

```
{
  "message": "Child profile not found"
}
```

2.5 Delete Child Profile

DELETE /api/v1/parent/child/:childId

Authentication: Required (Bearer Token - Parent only)

Description: Deletes child profile after OTP verification

Headers:

- Authorization: Bearer <JWT_TOKEN>

URL Parameters:

- childId - MongoDB ObjectId of the child

Request Body (JSON):

- ```
{
 "otp": "123456"
}
```

#### Success Response:

- Success Response (200):

```
{
 "message": "Child profile deleted successfully"
}
```

#### Error Responses:

- Error (400):

```
{
 "message": "Invalid OTP"
}
```

## 2.6 Get Academic Report

### **GET** /api/v1/parent/academic-report

**Authentication:** Required (Bearer Token - Parent only)

**Description:** Returns comprehensive academic report for children

#### Headers:

- Authorization: Bearer <JWT\_TOKEN>

### Query Parameters:

- `childId` (optional) - Returns report for specific child only

### Success Response:

- Success Response (200):

```
{
 "reports": [
 {
 "childId": "67890child001",
 "childName": "Emma",
 "classno": 1,
 "emoji": "🐱",
 "summary": {
 "totalQuizzes": 15,
 "completedCourses": 8,
 "avgScore": 85,
 "totalAttempts": 45
 },
 "subjects": [
 {
 "subjectId": "67890subject001",
 "subjectName": "Mathematics",
 "totalChapters": 10,
 "completedChapters": 8,
 "completionPercentage": 80,
 "quizCount": 8,
 "avgScore": 88
 }
]
 }
]
}
```

## 3. SUBJECT ROUTES

### 3.1 Add New Subject

**POST** `/api/v1/subject/add`

**Authentication:** None (Admin setup)

**Description:** Creates a new subject for a specific class

#### Headers:

- Authorization: Bearer <JWT\_TOKEN>

#### Request Body (JSON):

- ```
{
  "classnumber": 1,
  "name": "Mathematics",
```

```
    "price": 499
  }
}
```

Success Response:

- Success Response (201):

```
{
  "message": "Subject added successfully",
  "response": {
    "_id": "67890subject001",
    "classnumber": 1,
    "name": "Mathematics",
    "price": 499,
    "createdAt": "2026-01-16T10:30:00.000Z"
  },
  "usersInitialized": 5
}
```

Error Responses:

- Error (400):

```
{
  "message": "Subject already exists for this class"
}
```

3.2 Get All Subjects

GET /api/v1/subject/all/get

Authentication: None

Description: Returns all subjects across all classes

Success Response:

- Success Response (200):

```
{
  "message": "Subjects: ",
  "subjects": [
    {
      "_id": "67890subject001",
      "classnumber": 1,
      "name": "Mathematics",
      "price": 499
    },
    {
      "_id": "67890subject002",
      "classnumber": 1,
      "name": "English",
      "price": 399
    }
  ]
}
```

3.3 Get Subjects by Class

GET /api/v1/subject/by-class/:classno

Authentication: Required (Bearer Token)

Description: Returns subjects for a specific class with purchase status

Headers:

- Authorization: Bearer <JWT_TOKEN>

URL Parameters:

- classno - Class number (1, 2, 3, etc.)

Query Parameters:

- childId (optional) - Child's ID to check purchase status

Success Response:

- Success Response (200):

```
{
  "message": "Subjects retrieved successfully",
  "subjects": [
    {
      "_id": "67890subject001",
      "name": "Mathematics",
      "price": 499,
      "locked": false,
      "purchaseDate": "2026-01-15T08:00:00.000Z",
      "isPurchased": true
    }
  ]
}
```

Error Responses:

- Error (404):

```
{
  "message": "No subjects found for this class"
}
```

4. CHAPTER ROUTES

4.1 Add New Chapter

POST /api/v1/chapters/add

Authentication: None (Admin setup)

Description: Creates a new chapter for a subject

Headers:

- Authorization: Bearer <JWT_TOKEN>

Request Body (JSON):

- ```
{
 "subjectId": "67890subject001",
 "name": "Introduction to Numbers",
 "description": "Learn basic number concepts",
 "videourl": "/uploads/videos/number-magic.mp4"
}
```

#### Success Response:

- Success Response (201):

```
{
 "message": "Chapter added successfully",
 "response": {
 "_id": "67890chapter001",
 "subjectId": "67890subject001",
 "name": "Introduction to Numbers",
 "videourl": "/uploads/videos/number-magic.mp4"
 }
}
```

#### Error Responses:

- Error (400):

```
{
 "message": "Chapter already exists for this subject"
}
```

## 4.2 Get All Chapters

### GET /api/v1/chapters/all/get

**Authentication:** None

**Description:** Returns all chapters across all subjects

#### Success Response:

- Success Response (200):

```
{
 "message": "Chapters: ",
 "chapters": [
 {
 "_id": "67890chapter001",
 "name": "Introduction to Numbers",
 "videourl": "/uploads/videos/number-magic.mp4"
 }
]
}
```

### 4.3 Get Chapters by Subject

**GET** /api/v1/chapters/by-subject/:subjectId

**Authentication:** Required (Bearer Token)

**Description:** Returns chapters for a subject with completion status

#### Headers:

- Authorization: Bearer <JWT\_TOKEN>

#### URL Parameters:

- subjectId - MongoDB ObjectId of the subject

#### Query Parameters:

- childId (optional) - Child's ID to check progress

#### Success Response:

- Success Response (200):

```
{
 "chapters": [
 {
 "_id": "67890chapter001",
 "name": "Introduction to Numbers",
 "status": "completed",
 "isFirstChapter": true,
 "progress": 100
 },
 {
 "_id": "67890chapter002",
 "name": "Addition and Subtraction",
 "status": "locked",
 "progress": 0
 }
],
 "locked": false
}
```

#### Error Responses:

- Error (404):

```
{
 "message": "No chapters found for this subject"
}
```

### 4.4 Get Single Chapter

**GET** /api/v1/chapters/:chapterId

**Authentication:** Required (Bearer Token)

**Description:** Returns details of a specific chapter

**Headers:**

- Authorization: Bearer <JWT\_TOKEN>

**URL Parameters:**

- chapterId - MongoDB ObjectId of the chapter

**Query Parameters:**

- childId (optional) - Child's ID to check progress

**Success Response:**

- Success Response (200):

```
{
 "chapter": {
 "_id": "67890chapter001",
 "name": "Introduction to Numbers",
 "videourl": "/uploads/videos/number-magic.mp4",
 "status": "completed"
 },
 "locked": false
}
```

**Error Responses:**

- Error (403):

```
{
 "message": "Subject is locked. Please purchase."
}
```
- Error (404):

```
{
 "message": "Chapter not found"
}
```

## 4.5 Mark Chapter Progress

### **POST** /api/v1/chapters/:chapterId/progress

**Authentication:** Required (Bearer Token)

**Description:** Updates chapter progress (in-progress or completed)

**Headers:**

- Authorization: Bearer <JWT\_TOKEN>

**URL Parameters:**

- chapterId - MongoDB ObjectId of the chapter

#### Request Body (JSON):

- ```
{
  "childId": "67890child001",
  "completed": true
}
```

Success Response:

- Success Response (200):

```
{
  "message": "Chapter marked as completed",
  "progress": {
    "_id": "67890progress001",
    "userId": "67890child001",
    "completed": true
  }
}
```

4.6 Mark Chapter as Completed

POST /api/v1/chapters/:chapterId/complete

Authentication: Required (Bearer Token)

Description: Marks chapter as completed

Headers:

- Authorization: Bearer <JWT_TOKEN>

URL Parameters:

- chapterId - MongoDB ObjectId of the chapter

Request Body (JSON):

- ```
{
 "childId": "67890child001"
}
```

#### Success Response:

- Success Response (200):

```
{
 "message": "Chapter marked as completed"
}
```

## 5. PAYMENT ROUTES

### 5.1 Create Razorpay Order

**POST** /api/create-order

**Authentication:** None

**Description:** Creates a Razorpay order for payment

**Request Body (JSON):**

- ```
{
  "amount": 499
}
```

Success Response:

- Success Response (200):

```
{
  "id": "order_MNOPqrst123456",
  "amount": 49900,
  "currency": "INR",
  "status": "created"
}
```

Error Responses:

- Error (400):

```
{
  "message": "Amount is required and must be > 0"
}
```

5.2 Purchase Subject

POST /api/v1/payment/purchase-subject

Authentication: Required (Bearer Token)

Description: Purchases/unlocks a subject for the user

Headers:

- Authorization: Bearer <JWT_TOKEN>

Request Body (JSON):

- ```
{
 "subjectId": "67890subject001"
}
```

**Success Response:**

- Success Response (200):

```
{
```

```

 "success": true,
 "message": "Subject purchased successfully!",
 "subject": {
 "id": "67890subject001",
 "name": "Mathematics",
 "price": 499,
 "locked": false,
 "transactionId": "TXN_1737024000000"
 }
 }
}

```

#### Error Responses:

- Error (400):
 

```

{
 "message": "Subject already purchased"
}

```
- Error (404):
 

```

{
 "message": "Subject not found"
}

```

### 5.3 Get My Purchases

#### GET /api/v1/payment/my-purchases

**Authentication:** Required (Bearer Token)

**Description:** Returns all purchased subjects for the user

#### Headers:

- Authorization: Bearer <JWT\_TOKEN>

#### Success Response:

- Success Response (200):
 

```

{
 "purchases": [
 {
 "subjectId": {
 "name": "Mathematics",
 "price": 499
 },
 "locked": false,
 "purchaseDate": "2026-01-15T08:00:00.000Z"
 }
],
 "totalPurchases": 1
}

```

## 5.4 Check Subject Unlock Status

**GET** /api/v1/payment/check-unlock/:subjectId

**Authentication:** Required (Bearer Token)

**Description:** Checks if a subject is unlocked for the user

### Headers:

- Authorization: Bearer <JWT\_TOKEN>

### URL Parameters:

- subjectId - MongoDB ObjectId of the subject

### Success Response:

- Unlocked (200):

```
{
 "locked": false,
 "message": "Subject is unlocked",
 "purchaseDate": "2026-01-15T08:00:00.000Z"
}
```
- Locked (200):

```
{
 "locked": true,
 "message": "Subject is locked. Please purchase."
}
```

## 5.5 Get All Purchases (Admin)

**GET** /api/v1/payment/all-purchases

**Authentication:** None (Should be protected in production)

**Description:** Returns all purchases across all users

### Headers:

- Authorization: Bearer <JWT\_TOKEN>

### Success Response:

- Success Response (200):

```
{
 "purchases": [...],
 "totalPurchases": 10
}
```

## 5.6 Get Subscription Status

**GET** /api/v1/parent/subscription

**Authentication:** Required (Bearer Token - Parent only)

**Description:** Returns parent's subscription details

### Headers:

- Authorization: Bearer <JWT\_TOKEN>

### Success Response:

- Success Response (200):

```
{
 "status": "active",
 "type": "monthly",
 "startDate": "2026-01-01T00:00:00.000Z",
 "endDate": "2026-02-01T00:00:00.000Z"
}
```

Trial:

```
{
 "status": "trial",
 "type": null
}
```

## 5.7 Activate Subscription

**POST** /api/v1/parent/subscription

**Authentication:** Required (Bearer Token - Parent only)

**Description:** Activates subscription and unlocks all subjects for all children

### Headers:

- Authorization: Bearer <JWT\_TOKEN>

### Request Body (JSON):

- ```
{
  "type": "monthly",
  "razorpayPaymentId": "pay_MNOPqrst123456",
  "razorpayOrderId": "order_MNOPqrst123456",
  "razorpaySignature": "signature_hash"
}
```

Note: type can be "monthly" or "yearly"

Success Response:

- Success Response (200):

```
{
```

```

    "message": "Subscription activated successfully",
    "subscription": {
      "status": "active",
      "type": "monthly",
      "startDate": "2026-01-16T10:30:00.000Z",
      "endDate": "2026-02-16T10:30:00.000Z"
    },
    "childrenUnlocked": 2,
    "subjectsUnlocked": 6
  }
}

```

Error Responses:

- Error (400):


```

{
  "message": "Invalid subscription type"
}

```

6. QUIZ QUESTION ROUTES

6.1 Add Quiz Question

POST /api/v1/quiz/question/add

Authentication: None (Admin setup)

Description: Creates a new quiz question for a chapter

Headers:

- Authorization: Bearer <JWT_TOKEN>

Request Body (JSON):

- {


```

        "chapterId": "67890chapter001",
        "subject": "math",
        "topic": "Addition",
        "question": "What is 2 + 3?",
        "options": ["3", "4", "5", "6"],
        "correctAnswer": "5",
        "difficulty": "easy",
        "marks": 1
      
```

Note: difficulty can be "easy", "medium", or "hard"

Success Response:

- Success Response (201):


```

{
  "message": "Quiz question added successfully",
  "question": {
    "_id": "67890question001",
    "question": "What is 2 + 3?",

```

```

        "correctAnswer": "5",
        "difficulty": "easy"
    }
}

```

Error Responses:

- Error (400):


```

{
  "message": "Chapter ID, question, options, correct answer, subject, and
  topic are required"
}

```
- Error (404):


```

{
  "message": "Chapter not found"
}

```

6.2 Get Questions by Chapter (Student View)

GET /api/v1/quiz/questions/chapter/:chapterId

Authentication: None

Description: Returns questions for a chapter WITHOUT correct answers

URL Parameters:

- chapterId - MongoDB ObjectId of the chapter

Success Response:

- Success Response (200):


```

{
  "questions": [
    {
      "_id": "67890question001",
      "question": "What is 2 + 3?",
      "options": ["3", "4", "5", "6"],
      "difficulty": "easy"
    }
  ],
  "totalQuestions": 10,
  "totalMarks": 10
}

```

6.3 Get Questions by Chapter (Admin View)

GET /api/v1/quiz/questions/chapter/:chapterId/admin

Authentication: None (Should be protected in production)

Description: Returns questions WITH correct answers

Headers:

- Authorization: Bearer <JWT_TOKEN>

URL Parameters:

- chapterId - MongoDB ObjectId of the chapter

Success Response:

- Success Response (200):

```
{
  "questions": [
    {
      "_id": "67890question001",
      "question": "What is 2 + 3?",
      "correctAnswer": "5",
      "difficulty": "easy"
    }
  ]
}
```

6.4 Get Single Question

GET /api/v1/quiz/question/:questionId

Authentication: None

Description: Returns details of a specific question

URL Parameters:

- questionId - MongoDB ObjectId of the question

Success Response:

- Success Response (200):

```
{
  "question": {
    "_id": "67890question001",
    "question": "What is 2 + 3?",
    "correctAnswer": "5"
  }
}
```

Error Responses:

- Error (404):

```
{
  "message": "Question not found"
}
```

6.5 Update Question

PUT /api/v1/quiz/question/:questionId

Authentication: None (Should be protected in production)

Description: Updates an existing question

Headers:

- Authorization: Bearer <JWT_TOKEN>

URL Parameters:

- questionId - MongoDB ObjectId of the question

Request Body (JSON):

- ```
{
 "question": "What is 3 + 4?",
 "options": ["5", "6", "7", "8"],
 "correctAnswer": "7",
 "marks": 2
}
```

### Success Response:

- Success Response (200):  

```
{
 "message": "Question updated successfully"
}
```

## 6.6 Delete Question

**DELETE** /api/v1/quiz/question/:questionId

**Authentication:** None (Should be protected in production)

**Description:** Deletes a question

### Headers:

- Authorization: Bearer <JWT\_TOKEN>

### URL Parameters:

- questionId - MongoDB ObjectId of the question

### Success Response:

- Success Response (200):  

```
{
 "message": "Question deleted successfully"
}
```

### Error Responses:

- Error (404):

```
{
 "message": "Question not found"
}
```

## 7. QUIZ SCORE ROUTES

### 7.1 Save Quiz Score (Simple)

#### **POST** /api/v1/quiz/save-score

**Authentication:** Required (Bearer Token)

**Description:** Saves quiz score with simple score and total marks

### Headers:

- Authorization: Bearer <JWT\_TOKEN>

### Request Body (JSON):

- ```
{
  "childId": "67890child001",
  "chapterId": "67890chapter001",
  "score": 8,
  "totalMarks": 10
}
```

Success Response:

- Success Response (201):

```
{
  "message": "Quiz score saved successfully!",
  "result": {
    "score": 8,
    "totalMarks": 10,
    "percentage": 80,
    "quizScoreId": "67890quizscore001"
  }
}
```

Error Responses:

- Error (400):

```
{
  "message": "Child ID, Chapter ID, score, and totalMarks are required"
}
```
- Error (404):

```
{
  "message": "Child not found or you don't have permission"
}
```

7.2 Submit Quiz (With Answers)

POST /api/v1/quiz/submit

Authentication: Required (Bearer Token)

Description: Submits quiz answers, calculates and saves score

Headers:

- Authorization: Bearer <JWT_TOKEN>

Request Body (JSON):

- ```
{
 "childId": "67890child001",
 "chapterId": "67890chapter001",
 "answers": {
 "67890question001": "5",
 "67890question002": "12",
 "67890question003": "8"
 }
}
```

Note: Keys are question IDs, values are selected answers

### Success Response:

- Success Response (201):

```
{
 "message": "Quiz submitted successfully!",
 "result": {
 "score": 9,
 "totalMarks": 10,
 "percentage": 90,
 "quizScoreId": "67890quizscore001"
 }
}
```

### Error Responses:

- Error (404):

```
{
 "message": "No questions found for this chapter"
}
```

## 7.3 Get Child's Quiz Scores

**GET** /api/v1/quiz/scores/child/:childId

**Authentication:** Required (Bearer Token)

**Description:** Returns all quiz scores for a child

#### Headers:

- Authorization: Bearer <JWT\_TOKEN>

#### URL Parameters:

- childId - MongoDB ObjectId of the child

#### Success Response:

- Success Response (200):

```
{
 "child": {
 "name": "Emma",
 "classno": 1
 },
 "scores": [
 {
 "_id": "67890quizscore001",
 "chapterId": {
 "name": "Introduction to Numbers"
 },
 "score": 9,
 "totalMarks": 10,
 "percentage": 90
 }
],
 "totalQuizzes": 1
}
```

## 7.4 Get Quiz Scores for Specific Chapter

**GET** /api/v1/quiz/scores/child/:childId/chapter/:chapterId

**Authentication:** Required (Bearer Token)

**Description:** Returns all quiz attempts for a specific chapter

#### Headers:

- Authorization: Bearer <JWT\_TOKEN>

#### URL Parameters:

- childId - MongoDB ObjectId of the child
- chapterId - MongoDB ObjectId of the chapter

#### Success Response:

- Success Response (200):

```
{
 "scores": [
 {
 "score": 9,
 "totalMarks": 10,

```

```

 "percentage": 90,
 "createdAt": "2026-01-16T10:30:00.000Z"
 }
],
 "totalAttempts": 1
 }
}

```

#### Error Responses:

- Error (404):
 

```

{
 "message": "No quiz attempts found for this chapter"
}

```

## 7.5 Get Quiz Score by ID

**GET** /api/v1/quiz/score/:scoreId

**Authentication:** Required (Bearer Token)

**Description:** Returns detailed view of a specific quiz score

#### Headers:

- Authorization: Bearer <JWT\_TOKEN>

#### URL Parameters:

- scoreId - MongoDB ObjectId of the quiz score

#### Success Response:

- Success Response (200):
 

```

{
 "quizScore": {
 "_id": "67890quizscore001",
 "childId": {
 "name": "Emma"
 },
 "score": 9,
 "percentage": 90
 }
}

```

#### Error Responses:

- Error (403):
 

```

{
 "message": "You don't have permission to view this score"
}

```

## 7.6 Delete Quiz Score

**DELETE** /api/v1/quiz/score/:scoreId

**Authentication:** Required (Bearer Token)

**Description:** Deletes a quiz score

**Headers:**

- Authorization: Bearer <JWT\_TOKEN>

**URL Parameters:**

- scoreId - MongoDB ObjectId of the quiz score

**Success Response:**

- Success Response (200):

```
{
 "message": "Quiz score deleted successfully"
}
```

**Error Responses:**

- Error (404):

```
{
 "message": "Quiz score not found"
}
```

## 8. ADAPTIVE QUIZ ROUTES

### 8.1 Start Quiz

**GET** /api/quiz/start

**Authentication:** None

**Description:** Starts an adaptive quiz and returns the first question

**Query Parameters:**

- childId (required) - Child's ID
- subject (required) - Subject name (math, english, science)
- chapterId (optional) - Specific chapter ID

**Success Response:**

- Success Response (200):

```
{
 "completed": false,
 "question": {
 "_id": "67890question001",
 "question": "What is 2 + 3?",
 "options": ["3", "4", "5", "6"],
 "difficulty": "easy"
 }
}
```

```
Quiz Completed:
{
 "completed": true
}
```

#### Error Responses:

- Error (400):

```
{
 "message": "Missing childId or subject"
}
```

## 8.2 Submit Answer (Adaptive)

### POST /api/quiz/answer

**Authentication:** None

**Description:** Submits answer and returns next adaptive question

#### Request Body (JSON):

- ```
{
  "childId": "67890child001",
  "questionId": "67890question001",
  "isCorrect": true
}
```

Success Response:

- Success Response (200):

```
{
  "completed": false,
  "question": {
    "_id": "67890question005",
    "question": "What is 15 + 23?",
    "difficulty": "medium"
  },
  "adaptive": {
    "next_difficulty": "medium",
    "confidence": 0.75,
    "recommendation": "increase_difficulty"
  }
}
```

```
Quiz Completed:
{
  "completed": true
}
```

Error Responses:

- Error (404):

```
{
```

```
    "message": "Question not found"
  }
}
```

9. LEADERBOARD ROUTES

9.1 Get Leaderboard

GET `/api/v1/leaderboard`

Authentication: None

Description: Returns top 5 students and optional user position

Query Parameters:

- `userId` (optional) - User's ID to get their position

Success Response:

- Success Response (200):

```
{
  "top5": [
    {
      "rank": 1,
      "name": "Emma",
      "emoji": "🐱",
      "score": 1250
    },
    {
      "rank": 2,
      "name": "Noah",
      "emoji": "🐶",
      "score": 1180
    },
    {
      "rank": 3,
      "name": "Olivia",
      "emoji": "🐱",
      "score": 1050
    },
    {
      "rank": 4,
      "name": "Liam",
      "emoji": "🐻",
      "score": 980
    },
    {
      "rank": 5,
      "name": "Sophia",
      "emoji": "🐼",
      "score": 920
    }
  ],
}
```

```
"userPosition": {
  "rank": 12,
  "score": 650,
  "user": {
    "name": "Your Child",
    "emoji": "🐱"
  }
}
```

10. HEALTH CHECK

10.1 Health Check

GET /health

Authentication: None

Description: Health check endpoint to verify server is running

Success Response:

- Success Response (200):

```
{
  "status": "OK",
  "service": "Backend running"
}
```

APPENDIX

A. Authentication Flow

1. Parent Registration/Login:

- Call POST /api/v1/parent/login with email
- Receive OTP via email
- Call POST /api/v1/verify/parent with OTP
- Receive JWT token
- Use token in Authorization header for all protected routes

2. Token Format:

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

B. Common Error Responses

- 401 Unauthorized:
{"message": "Unauthorized"}
- 403 Forbidden:
{"message": "Parent access only"}
- 500 Internal Server Error:
{"message": "Error details here"}

C. MongoDB ObjectID Format

All IDs in the system use MongoDB ObjectID format:

- 24 hexadecimal characters
- Example: "67890abcdef123456789012"

D. Subject Names

Common subject names used in the system:

- math - Mathematics
- english - English
- science - Science

E. Difficulty Levels

Quiz questions support three difficulty levels:

- easy
- medium
- hard

F. Subscription Types

Two subscription types available:

- monthly - 1 month subscription
- yearly - 1 year subscription

G. Subscription Status

Three possible subscription statuses:

- trial - Free trial (default)
- active - Active paid subscription
- expired - Expired subscription